DOKUMEN DESAIN ARSITEKTUR SMART CONTRACT

Sistem Verifikasi Sertifikat & Ijazah Digital Berbasis Blockchain (CertiChain)

Mata Kuliah: Teknologi & Rekayasa Blockchain (Berbasis Proyek) | Dosen Pengampu: Mohamad Firdaus, M.Kom.

1. Pendahuluan & Latar Belakang

Sistem verifikasi ijazah dan sertifikat akademik konvensional saat ini masih menghadapi berbagai tantangan signifikan. Proses verifikasi manual yang dilakukan oleh instansi maupun pihak perusahaan (HRD) memakan waktu yang lama, memerlukan biaya administrasi, serta sangat bergantung pada otoritas terpusat. Di samping itu, tingkat pemalsuan dokumen fisik maupun dokumen digital dalam format PDF biasa semakin meningkat seiring dengan kemudahan manipulasi teknologi grafis.

Solusi berbasis Blockchain menawarkan pendekatan terdesentralisasi melalui pembuatan Smart Contract CertiChain pada jaringan Ethereum/EVM. Sistem ini mencatat kredensial ijazah dan pemilik dompet mahasiswa secara permanen (immutable) tanpa dapat diubah atau dimanipulasi oleh pihak mana pun setelah diterbitkan. Untuk mengoptimalkan penggunaan biaya transaksi (Gas Fee), file fisik PDF ijazah diolah menggunakan algoritma kriptografi menjadi bentuk Cryptographic Hash (IPFS CID / SHA-256), sehingga hanya sidik jari digital dokumen yang disimpan di dalam rantai blok.

2. Desain Entitas (Struct, Enum, & Mapping)

Arsitektur data dalam Smart Contract CertiChain dipetakan secara terstruktur untuk memodelkan objek dan status keabsahan dokumen akademik di dunia nyata:

2.1. Enum: StatusSertifikat

Enum digunakan untuk mengelola status siklus hidup dari sebuah ijazah/sertifikat:

- **Valid (0):** Menandakan sertifikat sah, terdaftar, dan diakui oleh instansi penerbit.
- **Dicabut (1):** Menandakan sertifikat telah dibatalkan atau dicabut keabsahannya (misalnya akibat pelanggaran akademik atau pembatalan gelar).

2.2. Struct: DataSertifikat

Struct memodelkan atribut utama dari dokumen ijazah:

- **namaPemilik (string):** Nama lengkap mahasiswa / penerima sertifikat.
- **programStudi (string):** Program studi atau bidang keahlian penerima.
- **hashDokumen (string):** Hash kriptografi (IPFS CID / SHA-256) dari file ijazah asli.
- **mahasiswa (address):** Alamat dompet (wallet) milik mahasiswa penerima ijazah.
- **tanggalTerbit (uint256):** Tanda waktu (timestamp) saat ijazah diterbitkan secara terenkripsi.
- **status (StatusSertifikat):** Status keabsahan dokumen (Valid / Dicabut).
- **alasanPencabutan (string):** Menampung alasan resmi jika ijazah dicabut secara on-chain.

- **waktuPencabutan (uint256):** Tanda waktu (timestamp) saat proses pencabutan dilakukan.

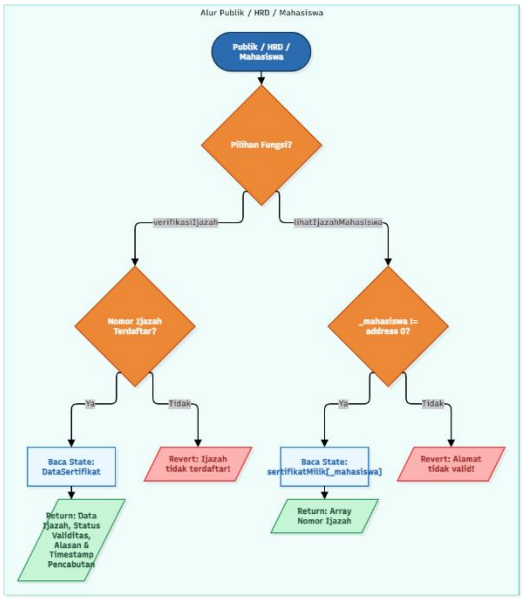
2.3. Mapping & Data State

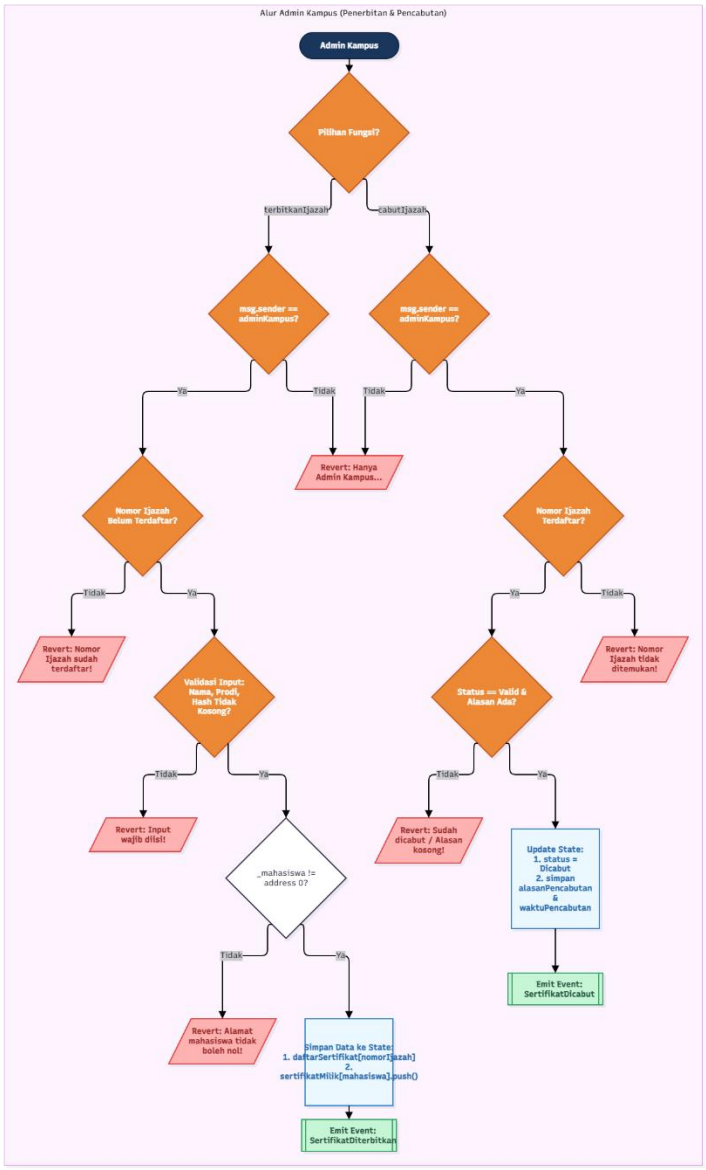
Penyimpanan state utama menggunakan dua struktur data mapping:

- **mapping(string => DataSertifikat) public daftarSertifikat:** Mengasosiasikan Nomor Ijazah Unik (string) sebagai Key langsung ke objek DataSertifikat (Struct) sebagai Value. Memberikan efisiensi pencarian data secara instan dengan kompleksitas waktu O(1).
- **mapping(address => string[]) private sertifikatMilik:** Mengasosiasikan Alamat Dompet Mahasiswa (address) sebagai Key ke daftar nomor ijazah (string[]) sebagai Value. Digunakan untuk melacak seluruh ijazah yang dimiliki oleh satu akun mahasiswa.

3. Flowchart Alur Transaksi & Interaksi System

Interaksi pengguna dengan Smart Contract dibagi menjadi dua alur utama: Alur Penerbitan & Pencabutan (Admin Kampus) dan Alur Verifikasi & Pengecekan (Publik / HRD / Mahasiswa).





3.1. Deskripsi & Penjelasan Alur Sistem

1. Alur Penerbitan Ijazah (Aktor: Admin Kampus)

- **Inisiasi Transaksi:** Admin Kampus memanggil fungsi terbitkanIjazah() dengan menginputkan nomor ijazah, nama pemilik, program studi, hash dokumen, dan alamat dompet mahasiswa.
- **Validasi Otorisasi (onlyAdmin):** Sistem mengecek msg.sender == adminKampus. Jika dipanggil selain admin, transaksi di-revert dengan pesan: "Error: Hanya Admin Kampus yang memiliki akses!".
- **Validasi Duplikasi & Input Kosong:** Sistem memastikan nomor ijazah belum terdaftar, serta memeriksa bahwa nomorIjazah, namaPemilik, programStudi, dan hashDokumen tidak boleh kosong, serta alamat mahasiswa bukan address(0).
- **Penyimpanan State & Logging:** Data ijazah disimpan ke daftarSertifikat, nomor ijazah di-push ke sertifikatMilik, dan sistem memancarkan Event SertifikatDiterbitkan.

2. Alur Pencabutan Ijazah (Aktor: Admin Kampus)

- **Inisiasi Pencabutan:** Admin memanggil fungsi cabutIjazah() mengisi nomor ijazah dan alasan pencabutan.
- **Validasi Pencabutan:** Memastikan nomor ijazah terdaftar, status ijazah masih Valid, dan alasan tidak boleh kosong.
- **Eksekusi & Logging:** Status diubah menjadi Dicabut, alasan & timestamp pencabutan dicatat on-chain, lalu memancarkan Event SertifikatDicabut.

3. Alur Verifikasi & Lihat Ijazah (Aktor: Publik / HRD / Mahasiswa)

- **Verifikasi Ijazah:** HRD/Publik memanggil fungsi read-only verifikasiIjazah(_nomorIjazah) untuk membaca data lengkap beserta status validitasnya.
- **Lihat Ijazah Mahasiswa:** Mahasiswa memanggil lihatIjazahMahasiswa(_mahasiswa) untuk mendapatkan seluruh daftar nomor ijazah miliknya secara free gas.

4. Kode Smart Contract (Solidity)

Berikut adalah implementasi lengkap Smart Contract CertiChain versi 2.0 (Solidity ^0.8.20):

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/**
 * @title CertiChain
 * @notice Sistem Verifikasi Sertifikat dan Ijazah Digital Berbasis Blockchain
 */
contract CertiChain {

    enum StatusSertifikat { Valid, Dicabut }

    struct DataSertifikat {
        string namaPemilik;
        string programStudi;
        string hashDokumen;
        address mahasiswa;
        uint256 tanggalTerbit;
        StatusSertifikat status;
        string alasanPencabutan;
        uint256 waktuPencabutan;
    }

    address public adminKampus;
    mapping(string => DataSertifikat) public daftarSertifikat;
    mapping(address => string[]) private sertifikatMilik;

    event SertifikatDiterbitkan(
        string indexed nomorIjazah,
        string namaPemilik,
        string programStudi,
        address indexed mahasiswa
    );

    event SertifikatDicabut(
```

```
        string indexed nomorIjazah,
        string alasan,
        uint256 waktu
    );

    modifier onlyAdmin() {
        require(msg.sender == adminKampus, "Error: Hanya Admin Kampus yang memiliki akses!");
        _;
    }

    constructor() {
        adminKampus = msg.sender;
    }

    function terbitkanIjazah(
        string memory _nomorIjazah,
        string memory _namaPemilik,
        string memory _programStudi,
        string memory _hashDokumen,
        address _mahasiswa
    ) external onlyAdmin {
        require(bytes(_nomorIjazah).length > 0, "Error: Nomor Ijazah tidak boleh kosong!");
        require(bytes(daftarSertifikat[_nomorIjazah].namaPemilik).length == 0, "Error: Nomor Ijazah sudah terdaftar!");
        require(bytes(_namaPemilik).length > 0, "Nama pemilik wajib diisi");
        require(bytes(_programStudi).length > 0, "Program studi wajib diisi");
        require(bytes(_hashDokumen).length > 0, "Hash dokumen wajib diisi");
        require(_mahasiswa != address(0), "Error: Alamat mahasiswa tidak boleh kosong!");

        daftarSertifikat[_nomorIjazah] = DataSertifikat({
            namaPemilik: _namaPemilik,
            programStudi: _programStudi,
            hashDokumen: _hashDokumen,
            mahasiswa: _mahasiswa,
            tanggalTerbit: block.timestamp,
            status: StatusSertifikat.Valid,
            alasanPencabutan: "",
            waktuPencabutan: 0
        });

        sertifikatMilik[_mahasiswa].push(_nomorIjazah);

        emit SertifikatDiterbitkan(_nomorIjazah, _namaPemilik, _programStudi, _mahasiswa);
    }

    function cabutIjazah(string memory _nomorIjazah, string memory _alasan) external onlyAdmin {
        require(bytes(daftarSertifikat[_nomorIjazah].namaPemilik).length > 0, "Error: Nomor Ijazah tidak ditemukan!");
        require(daftarSertifikat[_nomorIjazah].status == StatusSertifikat.Valid, "Error: Ijazah sudah dalam status
dicabut!");
        require(bytes(_alasan).length > 0, "Error: Alasan pencabutan wajib diisi!");

        daftarSertifikat[_nomorIjazah].status = StatusSertifikat.Dicabut;
        daftarSertifikat[_nomorIjazah].alasanPencabutan = _alasan;
        daftarSertifikat[_nomorIjazah].waktuPencabutan = block.timestamp;

        emit SertifikatDicabut(_nomorIjazah, _alasan, block.timestamp);
    }

    function verifikasiIjazah(string memory _nomorIjazah) external view returns (
```

```
        string memory namaPemilik,
        string memory programStudi,
        string memory hashDokumen,
        address mahasiswa,
        uint256 tanggalTerbit,
        bool isValid,
        string memory alasanPencabutan,
        uint256 waktuPencabutan
    ) {
        require(bytes(daftarSertifikat[_nomorIjazah].namaPemilik).length > 0, "Error: Ijazah tidak terdaftar di sistem!");

        DataSertifikat memory cert = daftarSertifikat[_nomorIjazah];
        bool statusValid = (cert.status == StatusSertifikat.Valid);

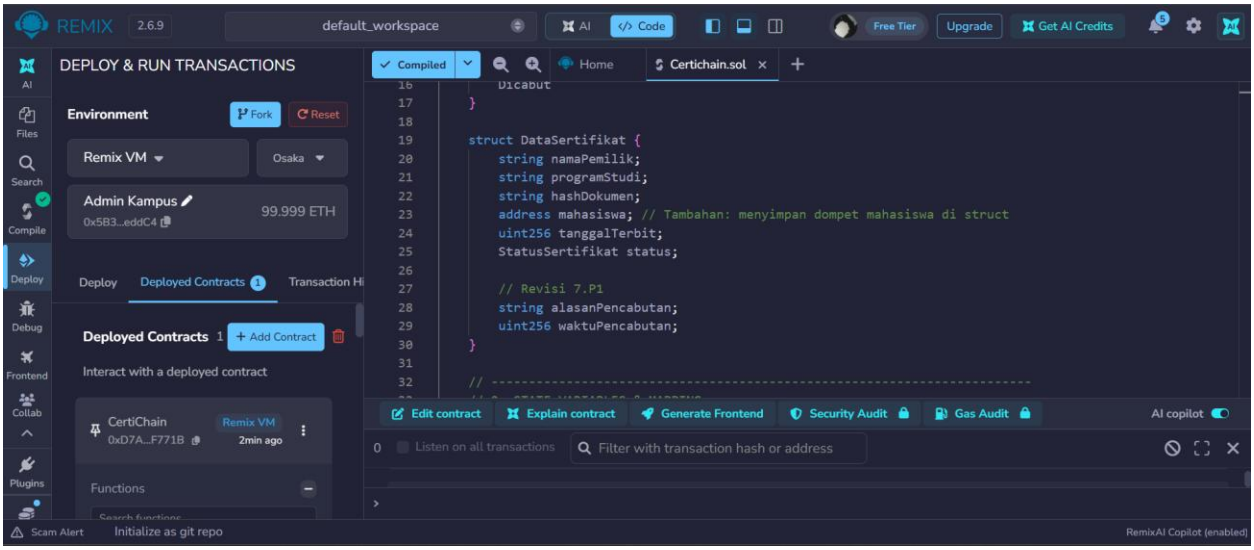
        return (
            cert.namaPemilik,
            cert.programStudi,
            cert.hashDokumen,
            cert.mahasiswa,
            cert.tanggalTerbit,
            statusValid,
            cert.alasanPencabutan,
            cert.waktuPencabutan
        );
    }

    function lihatIjazahMahasiswa(address _mahasiswa) external view returns (string[] memory) {
        require(_mahasiswa != address(0), "Error: Alamat mahasiswa tidak valid!");
        return sertifikatMilik[_mahasiswa];
    }
}
```

5. Hasil Pengujian & Simulasi (Live Demo)

5.1. Pengujian Compilation & Deployment Success

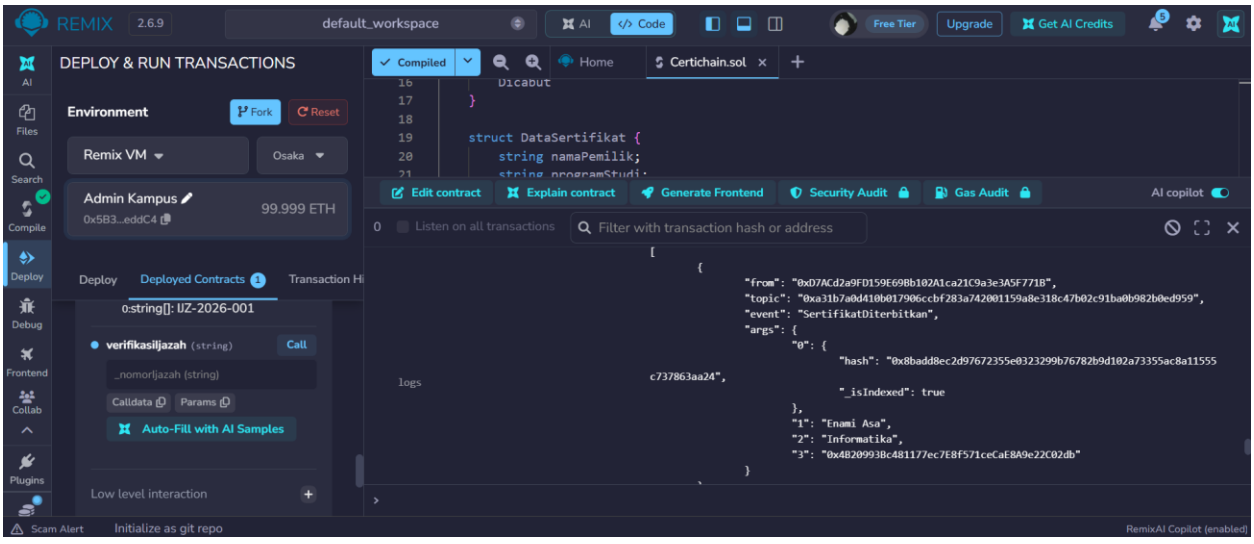
Proses kompilasi dan penyebaran (deploy) kontrak dilakukan di Remix IDE menggunakan compiler version 0.8.20.

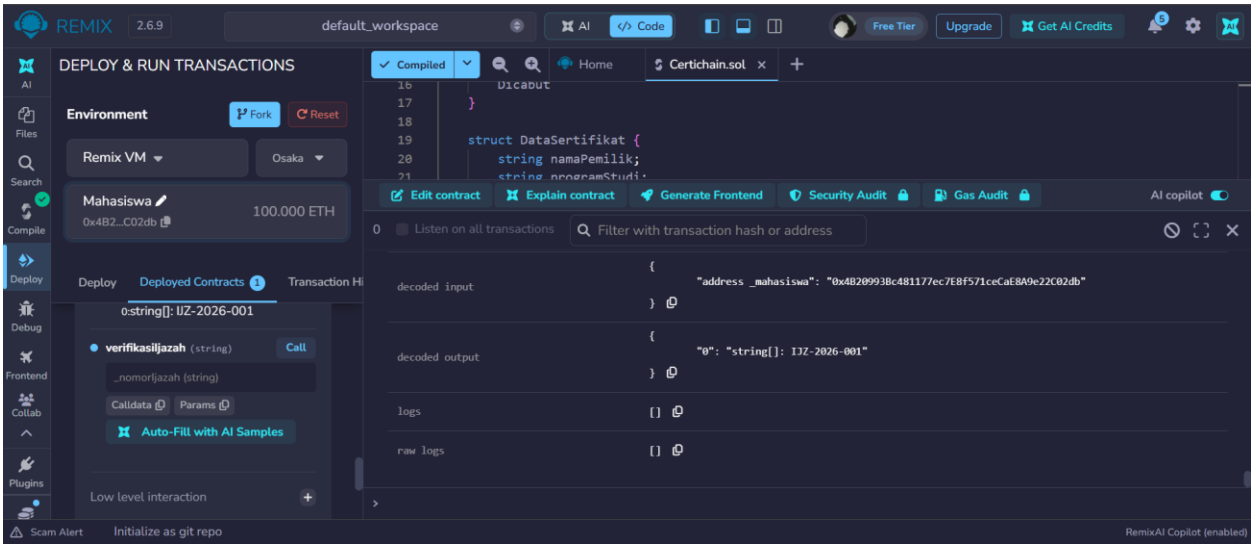


5.2. Pengujian Alur Normal (Penerbitan & Lihat Ijazah Mahasiswa)

Pengujian dilakukan menggunakan Akun Admin untuk menerbitkan ijazah dengan alamat mahasiswa (Account 2). Selanjutnya, Akun 2 memanggil fungsi lihatIjazahMahasiswa dan verifikasiIjazah.

- **Nomor Ijazah:** "IJZ-2026-001"
- **Nama Pemilik:** "Enami Asa"
- **Program Studi:** "Informatika"
- **Hash Dokumen:** "QmXoypizjW3WknFiJnKLwHCnL72vedxjQkDDP1mXWo6uco"
- **Alamat Mahasiswa:** "0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2" (Account 3)

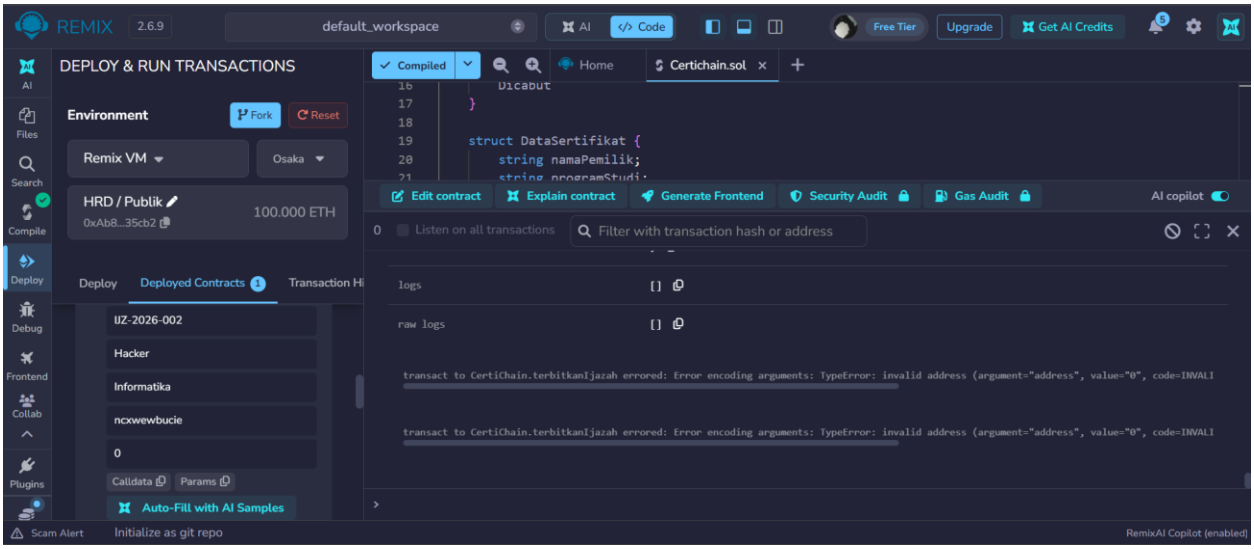




5.3. Pengujian Keamanan & Simulasi Serangan (Revert Check)

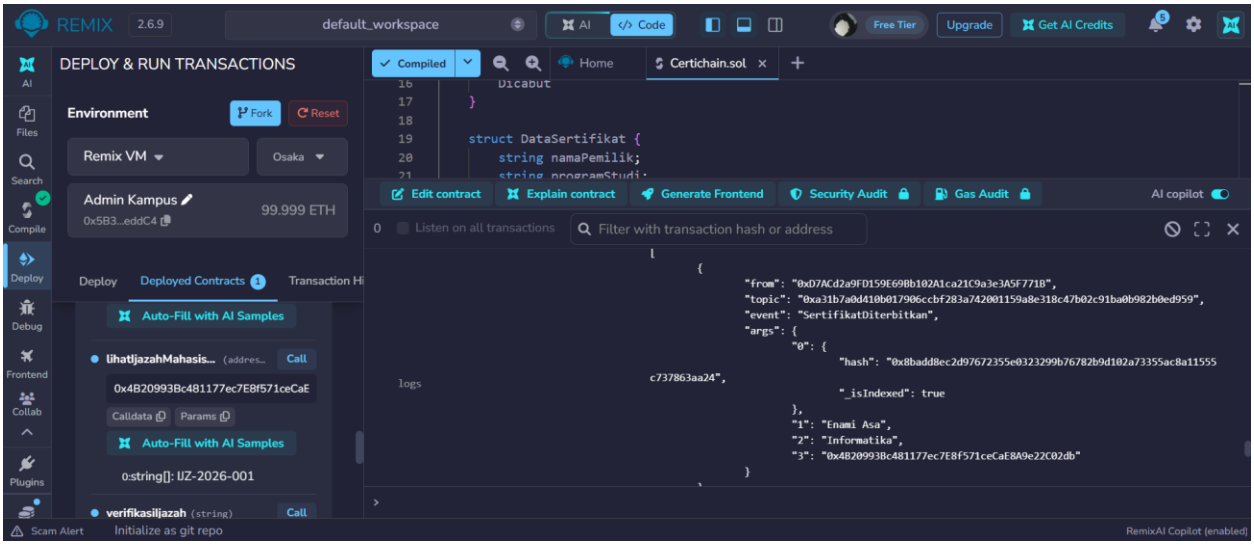
Simulasi serangan dilakukan dengan menguji kondisi kegagalan (revert check):

- **Serangan Otorisasi:** Mengubah akun aktif menjadi Akun 2 (HRD / Publik) lalu mencoba memanggil terbitkanIjazah().
- **Input Ilegal/Kosong:** Memanggil terbitkanIjazah() dengan mengosongkan nama/prodi/hash atau mengisi alamat mahasiswa dengan 0x0000000000000000000000000000000000000000 (address(0)).



5.4. Pengujian Logging (Event Execution)

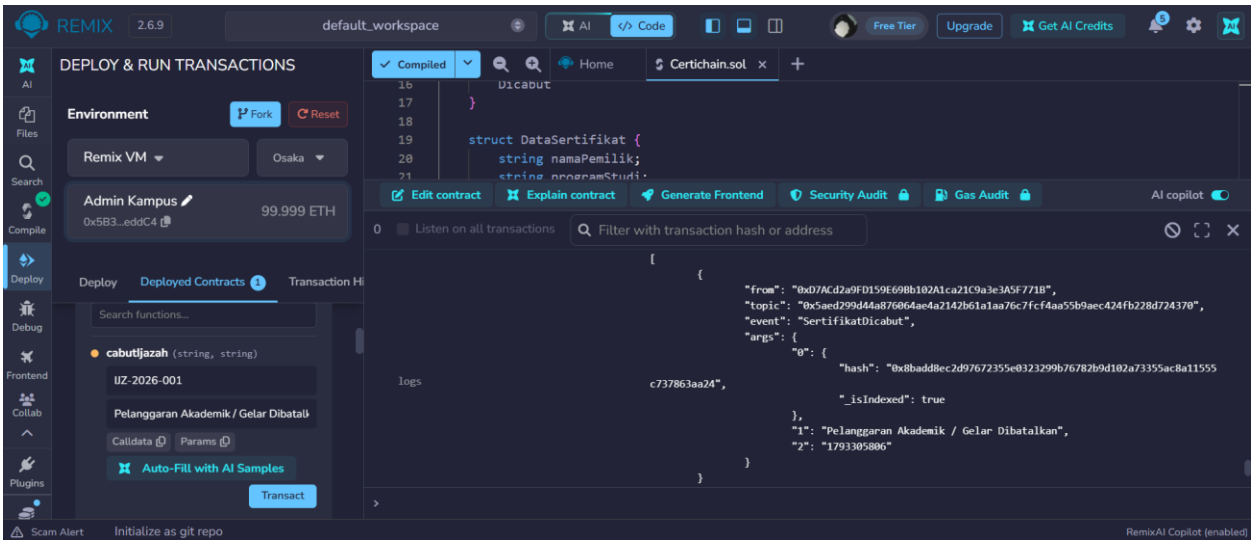
Setiap transaksi penerbitan dan pencabutan memancarkan Event ke log blockchain untuk memfasilitasi audit transaksi secara efisien.

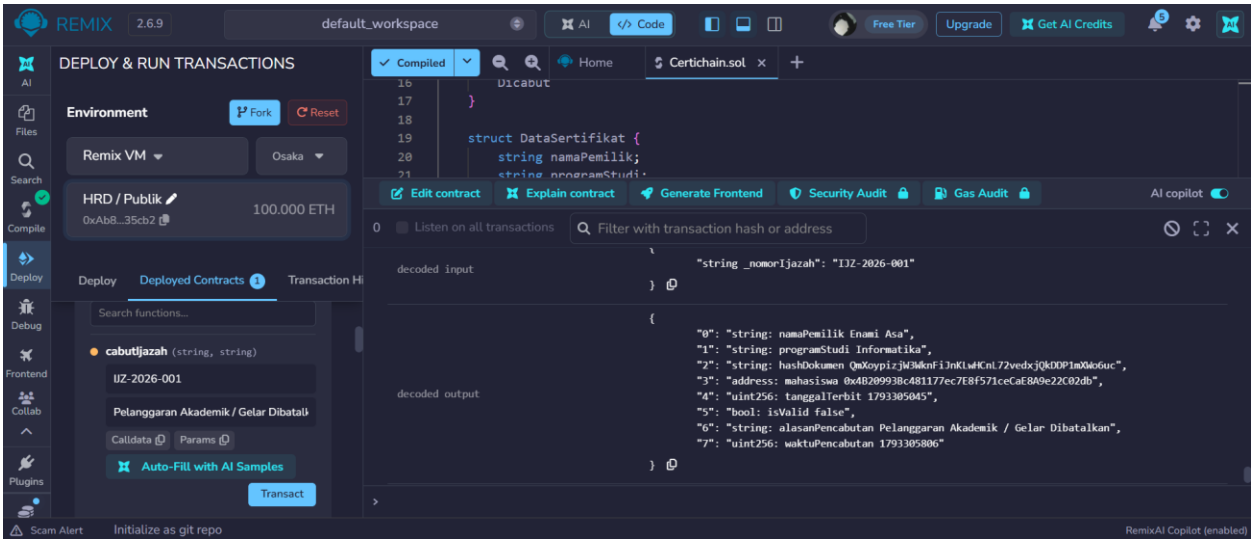


5.5. Pengujian Pencabutan & Verifikasi Ulang (Cabut Ijazah)

Admin Kampus mencabut ijazah "IJZ-2026-001" dengan memanggil cabutIjazah dan memberikan alasan "Pelanggaran Akademik / Gelar Dibatalkan". Selanjutnya, pihak HRD/Publik memanggil fungsi verifikasiIjazah("IJZ-2026-001").

- **Hasil isValid:** false (Dokumen dinyatakan tidak lagi valid)
- **Alasan Pencabutan:** "Pelanggaran Akademik / Gelar Dibatalkan"
- **Waktu Pencabutan:** Unix timestamp pencabutan tersimpan secara rinci.





6. Kesimpulan

Smart Contract CertiChain versi 2.0 ini telah memenuhi seluruh kriteria kelayakan fundamental Web3 serta menyelesaikan seluruh butir revisi UTS. Sistem terbukti aman dari potensi manipulasi data melalui proteksi role onlyAdmin, tervalidasi penuh terhadap serangan input kosong atau alamat nol, efisien dalam pemanfaatan gas fee berkat struktur event dan fungsi read-only, serta transparan dalam pencatatan status dan alasan pencabutan ijazah secara on-chain. Smart Contract ini telah siap diintegrasikan pada arsitektur dApp skala produksi.